

Using interrupts and data transfer between microcontrollers with the help of the serial interface

1 Paper's purpose

By the end of this first laboratory every student should have basic knowledge regarding interrupts technique and their use in the communication between two microcontrollers.

2 Introduction

In this chapter will be presented:

2.1. Interrupts sources provided by the microcontroller

2.1 Interrupts

The CIP51 core provides the user with an extended interrupts system with 20 sources, with two priority levels. Each interrupt is associated with a flag from the SFR registers. When the conditions for a valid interruption are met, this flag is set.

If interrupts are enabled for a particular source, an interrupt request (IRQ) is generated when the interrupt flag is set. As soon as the execution of the current instruction ends, the processing core generates a call (LCALL) to an address in the code memory associated with the interrupt, to execute the interrupt service routine (ISR). Each ISR must end with a RETI instruction, which takes the program to the next instruction that would have been executed if the interrupt had not occurred. If interrupts are not enabled, the interrupt flag setting is ignored and the program is executed as usual. The interrupt flag is set whether the interrupt is enabled or disabled.

Each interrupt can be enabled or disabled separately with the associated enable bit located in a SFR (IE-EIE2). However, interrupt requests must be enabled globally by setting IE.7 bit (EA-Interrupt Enable Bit) before the individual interrupts are recognized by the processing core. Resetting EA bit disables all interrupts regardless of their own enable/disable status.

Warning: Any instruction that resets the EA bit („CLR EA”) must be followed **immediately** by an instruction whose format has at least two bytes. If an interrupt occurs while the EA bit reset instruction is executed and the next instruction takes only one clock cycle, it may be considered valid and the interrupt service routine is executed. However, reading the EA bit will return the value 0 inside the routine. When the EA reset instruction is followed by a multi-cycle instruction, the interrupt is not considered valid.

Some of the interrupt flags are hardware reset as soon as the execution of the interrupt service routine is started. But most of them must be software reset before ending ISR. If a flag remains set after the RETI instruction has been executed, a new interrupt request will be generated immediately and the processing core will revert to the ISR after the current instruction is completed.

2.1.1 Interrupts processing

When an interruption takes place, the microcontroller executes the following steps before serving the received interrupt request:

- The current instruction ends its execution.
- The program counter is saved in the stack, and the SFR page is saved in the hardware stack for the SFR pages.
- The current state of the interruptions is saved internally.
- Interrupts with the same priority are blocked.
- The address of the interrupt vector is loaded into the program counter.
- The interrupt service routine is executed.

- The interrupt service routine ends with RETI instruction.
- The SFR_PAGE is extracted from the hardware stack, the program address saved in the software stack of the microcontroller is loaded in the PC.
- Interrupts with the same priority are activated.

2.1.2 Interrupt sources and vectors

The microcontroller has 20 interrupt sources. An interrupt can be software simulated by setting the interrupt flag. If the interrupt is enabled, an IRQ will be generated and the processor will execute the ISR from the address associated with that interrupt. Interrupt sources, associated vector addresses, order of priorities and control bits are shown in Table 12.4 (file C8051F04xRev1_4.pdf, p. 155).

2.1.3 Priorities setting

Each interrupt source can have two priority levels: low and high. A low priority interrupt can be temporarily disabled by a high priority interrupt. A high priority interrupt can not be disabled temporarily by another interrupt. Each interrupt has a priority setting bit in the SFR registers (IE-EIP2). The predefined priority is the low one. If two interrupt requests occur at the same time, the high priority interrupt is served. If the two interrupts have the same priority, a default order (see Table 12.4 of the file C8051F04xRev1_4.pdf, pages 155-156) is used for the service decision.

2.1.4 Interrupts latency

Response time for an IRQ depends on the processor state at that time. Pending interruptions are checked and decoded according to their priority at each clock cycle. The shortest response time has five clock cycles: one to detect the interruption and four to execute the ISR call instruction (LCALL). If an interrupt is pending when the RETI instruction is executed, a single instruction is executed before the call (LCALL) of the interrupt service routine. Thus, the longer response time for an interrupt takes place when the processor executes the RETI instruction followed by a DIV instruction. In this case, the response time is 18 clock cycles: one to detect the interrupt, five to execute the RETI instruction, one to execute the DIV instruction, and four to execute the LCALL instruction. If the processor executes an ISR for an interrupt with equal or higher priority, the new interrupt will not be served until the current ISR execution is finished, including the RETI instruction and the following instruction.

The special registers used to configure interrupts are presented in the tables on the following pages (file C8051F04xRev1_4.pdf, pages 158-163).

3 Application example

3.1 Requirements

Implement a voice communication system between two C8051F040 microcontrollers. It is required to use the serial interface and its interrupt system.

3.2 Proposed solution

3.2.1 Hardware module description

We will briefly present the implementation of a system that uses two C8051F040 development boards and four external modules (two inputs, two outputs) identical to the ones used in Paper 4. For the hardware description of these modules, see Paper 4, Chapter 3.2.1.

The two development boards are connected via a serial cross cable.

3.2.2 Algorithm description

Note that the algorithm and the source code follow the general structure presented in Paper 1, chapter 3.2.2.

The solution presented will be composed of several procedures: `main`, `init` and `UART1Irr`, which is the interrupt service routine of the interrupt generated by the serial interface.

The main program (`main`) calls the initialization subroutine (`init`) then passes to the area labeled `start`. In the area labeled `start` an interrupt that primes the communication is forced. The `jmp $` instruction follows, representing an infinite loop that does not execute anything.

The infinite loop is interrupted when an event occurs (reception, end of transmission). In this case, the PC (Program Counter) is initialized with the value of the interrupt vector corresponding to UART1 (0x00A3). At this address there is a jump instruction to the interrupt service routine (`UART1Irr`).

`UART1Irr` routine tests the interrupt type (reception, end of transmission) and, consequently, passes either in the labeled `send` area or in the labeled `receive` area. At the beginning of the routine, the context is saved (SFRPAGE, ACC). At the end of the routine the context is restored.

The area labeled `receive` retrieves the last data byte received and transfers it to DAC0. Thus, the signal sample is converted from digital into analog and played back in the speaker. Also, in this area, the RI1 interrupt flag is reset (UART1 is ready for a new reception).

The area labeled `send` takes a sample from the analog signal from the microphone via ADC2. The converted sample is read from the ADC2 and written in the buffer register of UART1. That's how its transmission starts. Then, a new conversion is forced (reading new data from the microphone); these data will be transmitted to the next call of the interrupt service routine. Note: The TI1 interrupt flag is reset before the transmission starts (the end of the transmission will cause its setting and therefore a new interruption).

The initialization subroutine

- disables the interrupt system
 - please see Paper 1 for details.
- ul disables the watchdog timer mechanism
 - please see Paper 1 for details.
- enables the crossbar
 - please see Paper 1 for details.
- configures port P1
 - please see Paper 2 for details.
- configures the reference voltage
 - please see Paper 2 for details.
- configures the analog-to-digital converter
 - please see Paper 2 for details.
- configures the digital-to-analog converter
 - please see Paper 4 for details.
- configures the internal oscillator
 - please see Paper 4 for details.

- configures Timer1
 - please see Paper 3 for details.
- configures UART1
 - please see Paper 3 for details.

4 Exercises

1. Create a new project using the application from Appendix 5.1
2. Complete the empty spaces in the program with the instructions required for the configurations mentioned in the code comments.
3. Execute the complete code step by step, tracking the effects on the registers by placing a breakpoint on the interrupt vector.
4. Modify the program from Paper 4, Appendix 5.2, to use Timer3 interrupt in order to start an ADC2 conversion and DAC0 transfer.
5. Write a program to modify Appendix 5.2 of Paper 4 such that:
 - ADC2 starts the conversion at Timer 2 overflow,
 - transfer to DAC0 is performed inside an interrupt vector generated by the end of ADC2 conversion
6. Write a program that generates a rectangular signal with the frequency controlled by A and D keys so that the frequency is doubled (one octave) by pressing the D key and reduced to half by pressing the A key, using timers interrupts.

5 Appendixes

5.1 voiceTransmission source code

```

;-----
;
; FILE NAME   : voiceTransmission.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : interMicroProcessor voice transmission.
;
; NOTES:
;
;-----
; EQUATES
;-----

                $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg      AT 0x0000                    ; Reset Vector
                ljmp      main                        ; Locate a jump to the start of code
                                                    ;at the reset vector.
                cseg      AT 0x00A3                    ; UART1 interrupt vector
                ljmp      UART1Irr                    ; Jump at the interrupt response routine

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment      CODE

                rseg      mainCodeSeg                ; Switch to this code segment.
                using     0                          ; Specify register bank for the
                                                    ;following program code.

main:          call      init

start:         setb      TI1                          ; Trigger the first interrupt
                jmp      $                          ; Wait for interrupts

;-----
; FUNCTION CODE
;-----

init:          ...                                ; Disable global interrupts
                ...                                ; Disable Watch Dog Timer
                ...

initIOAndCross: ...                                ; Use SFRs in the
                                                    ;configuration Page
                ...                                ; Enable the crossbar and UART 1
                ...                                ; Configure P1.0 as analog input

initVREF:      ...                                ; Use SFRs on the 0x00 Page
                ...                                ; ADC2 voltage reference from internal VREF
                                                    ; Enable Bias Generator

initADC2:      ...                                ; Use SFRs in the ADC2 Page
                ...                                ; Set AIN1.0 as single-ended input
                ...                                ; Select AIN1.0 as input channel for
                                                    ;the analog multiplexer
                ...                                ; Configure a *1 Gain
                ...                                ; Enable ADC2; Continuous tracking;

initDAC0:      ...                                ; Use SFRs on the DAC0 Page
                ...                                ; Enable DAC0
                ...                                ; a conversion will start only after
                ...                                ; a write in DAC0H; the data format used is lxx.

initOscillator: ...                                ; set SYSCLK to 24.5 MHz

initTimer1:    ...                                ; Use SFRs in Timer 1 Page
                ...                                ; Use Timer 1 in 8-bit auto-reload mode
                ...                                ; Set the reload value

```

```

...                                ; Timer 1 will use the system clock
...                                ; Enable Timer 1

initUART1:                        ; Use SFRs in UART 1 Page
...                                ; Use UART 1 in 8-bit mode

initInterrupts:                   ; Enable and set high priority for
    mov     EIE2, #0x40           ; UART1 transmit/receive interrupt
    mov     EIP2, #0x40           ; Enable all interrupts
    setb    EA
    ret

;-----
; INTERRUPT SERVICE ROUTINES
;-----

UART1Irr:                         ; Save the context (SFRPAGE and ACC)
    push    SFRPAGE
    push    ACC
    jnb     RI1, send              ; Check the interrupt source:
                                   ;end of transmission or receive

    receive:                       ; If data received, send it to DAC0
        mov     SFRPAGE, #UART1_PAGE
        mov     A, SBUF1
        clr     RI1               ; Clear the interrupt flag
        mov     SFRPAGE, #DAC0_PAGE
        mov     DAC0H, A          ; Write the data in DAC0H
        jmp     exit

    send:                          ; If data sent, read another byte
        mov     SFRPAGE, #ADC2_PAGE
        mov     A, ADC2           ;from the ADC2
        clr     AD2INT            ; Force a new conversion
        setb    AD2BUSY
        mov     SFRPAGE, #UART1_PAGE
        clr     TI1              ; Clear the interrupt flag
        mov     SBUF1, A          ; Sent the read data through the serial
                                   ;connection

    exit:                          ; Restore the context (SFRPAGE and ACC)
        pop     ACC
        pop     SFRPAGE
        reti

;-----
; End of file.

```